PROJECT WALKTHROUGH

How I Deployed a Static Portfolio Website on AWS S3

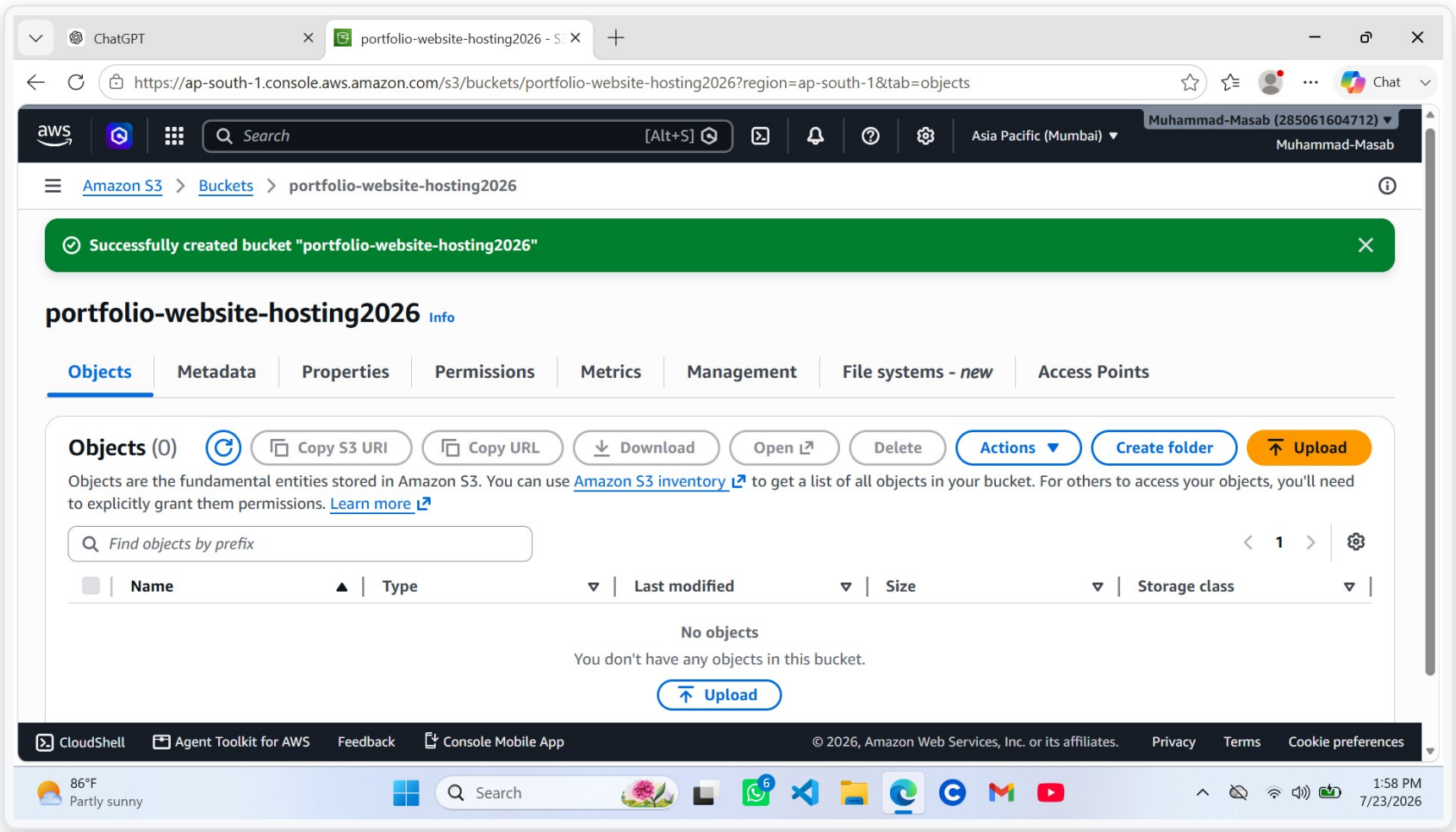
A step-by-step look at what I did, why I did it, and how I fixed a real deployment error along the way.

SERVICE	Amazon S3
REGION	ap-south-1 (Mumbai)
BUCKET	portfolio-website-hosting2026

SWIPE TO SEE EACH STEP →

STEP 1 OF 5

Creating the S3 Bucket



WHAT

Created a new S3 bucket named portfolio-website-hosting2026 in the Asia Pacific (Mumbai) region.

WHY

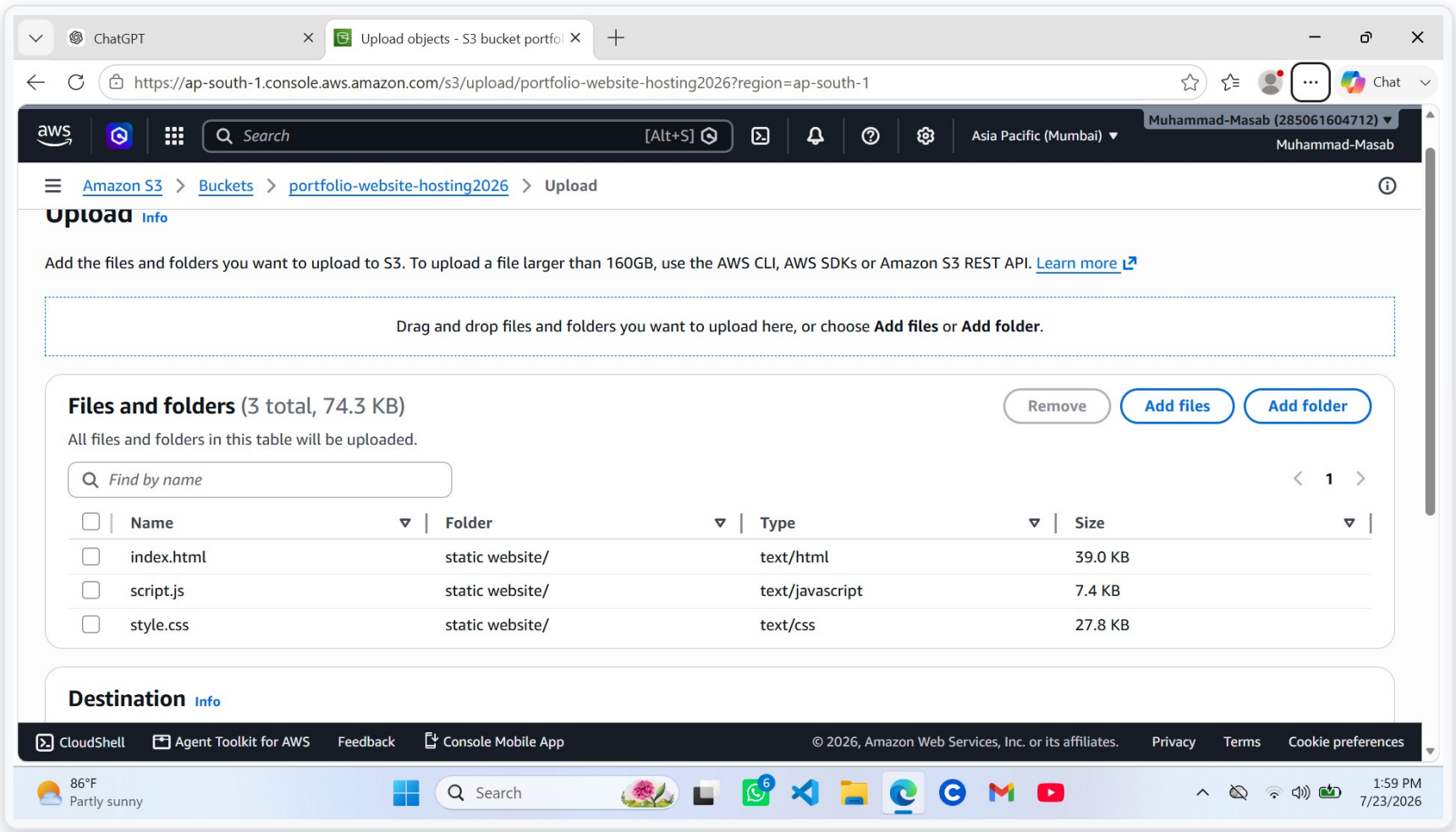
Every S3 object needs a home. The bucket is the storage container that will hold and later serve the website's files.

HOW

AWS Console → S3 → Create bucket → entered a globally unique name → selected ap-south-1 as the region → confirmed defaults.

STEP 2 OF 5

Uploading the Website Files



WHAT

Uploaded index.html, style.css, and script.js (74.3 KB total) to the bucket.

WHY

These three files are the actual portfolio site — the markup, styling, and interactivity a visitor's browser needs to render.

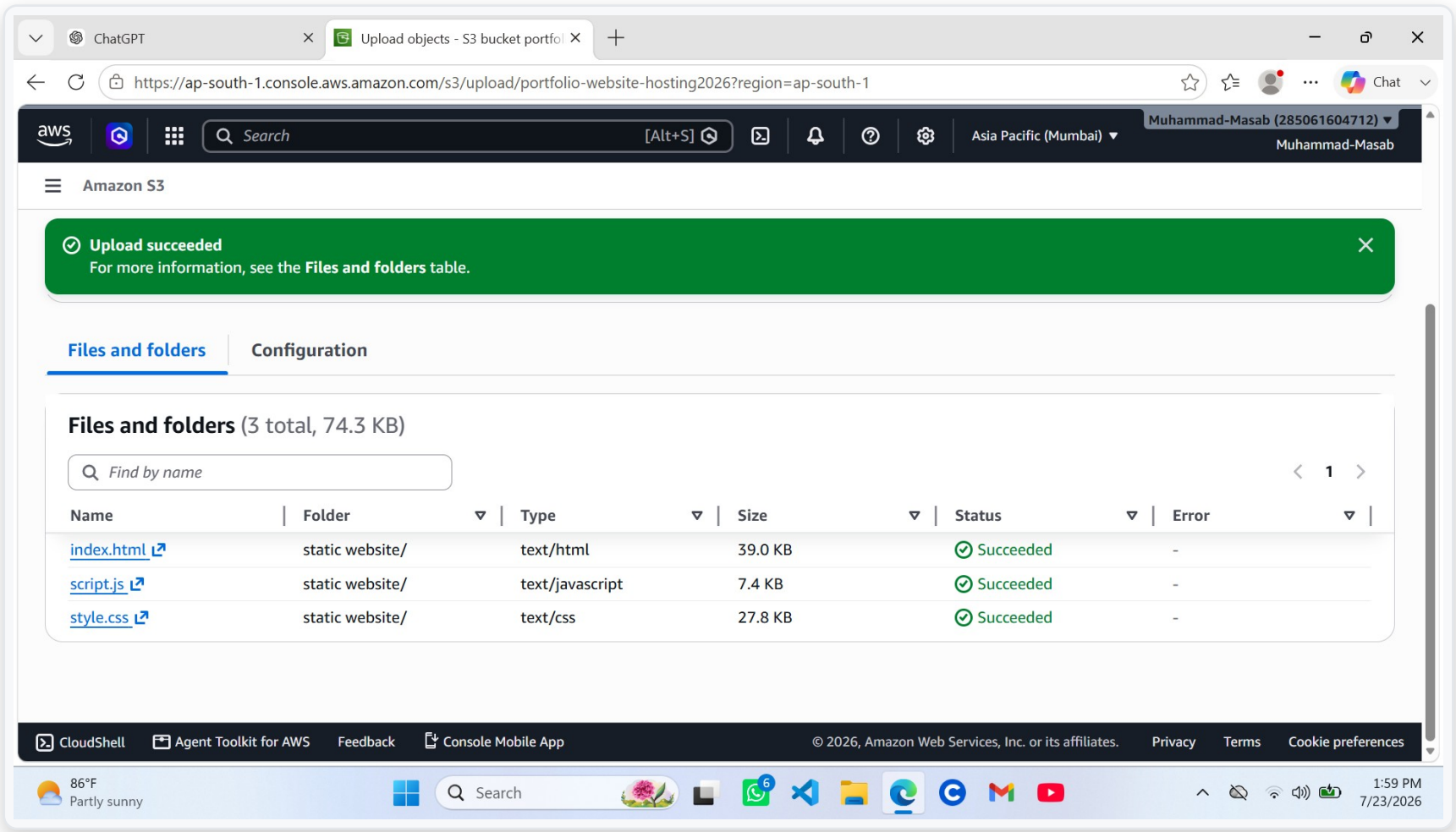
HOW

Used the S3 console's Upload screen, added the files via Add files, and reviewed the file list before submitting.

⚠️ *These files were staged inside a "static website/" folder here — a choice that came back to bite me in Step 5.*

STEP 3 OF 5

Confirming a Successful Upload



WHAT

Verified that all three files uploaded with a "Succeeded" status and zero errors.

WHY

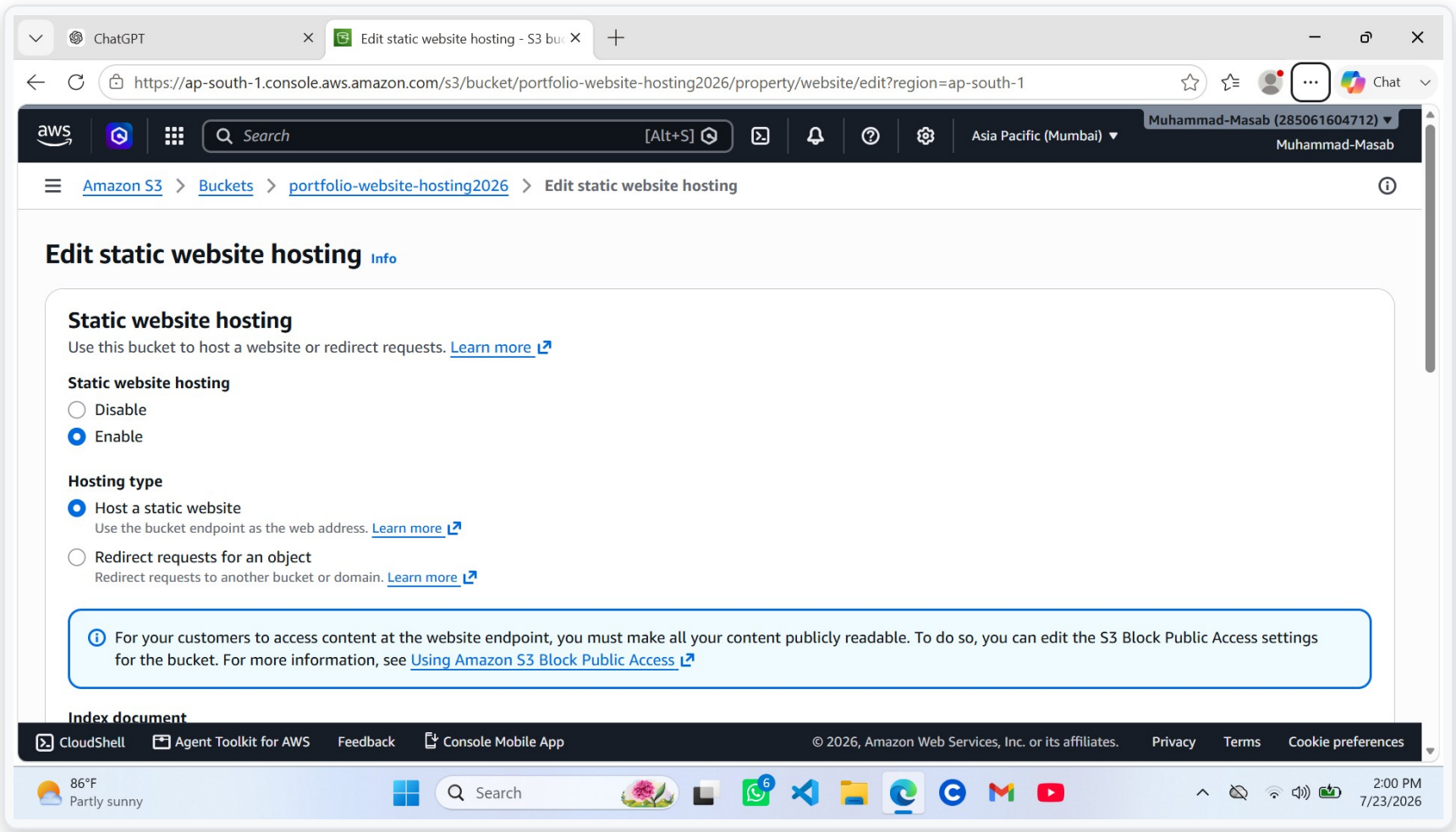
Before touching any hosting settings, I wanted certainty the files actually landed in the bucket and weren't silently dropped.

HOW

AWS surfaces a per-file status table after every upload — I checked each row instead of trusting the generic success banner alone.

STEP 4 OF 5

Enabling Static Website Hosting



WHAT

Turned on the Static website hosting property and set index.html as the index document.

WHY

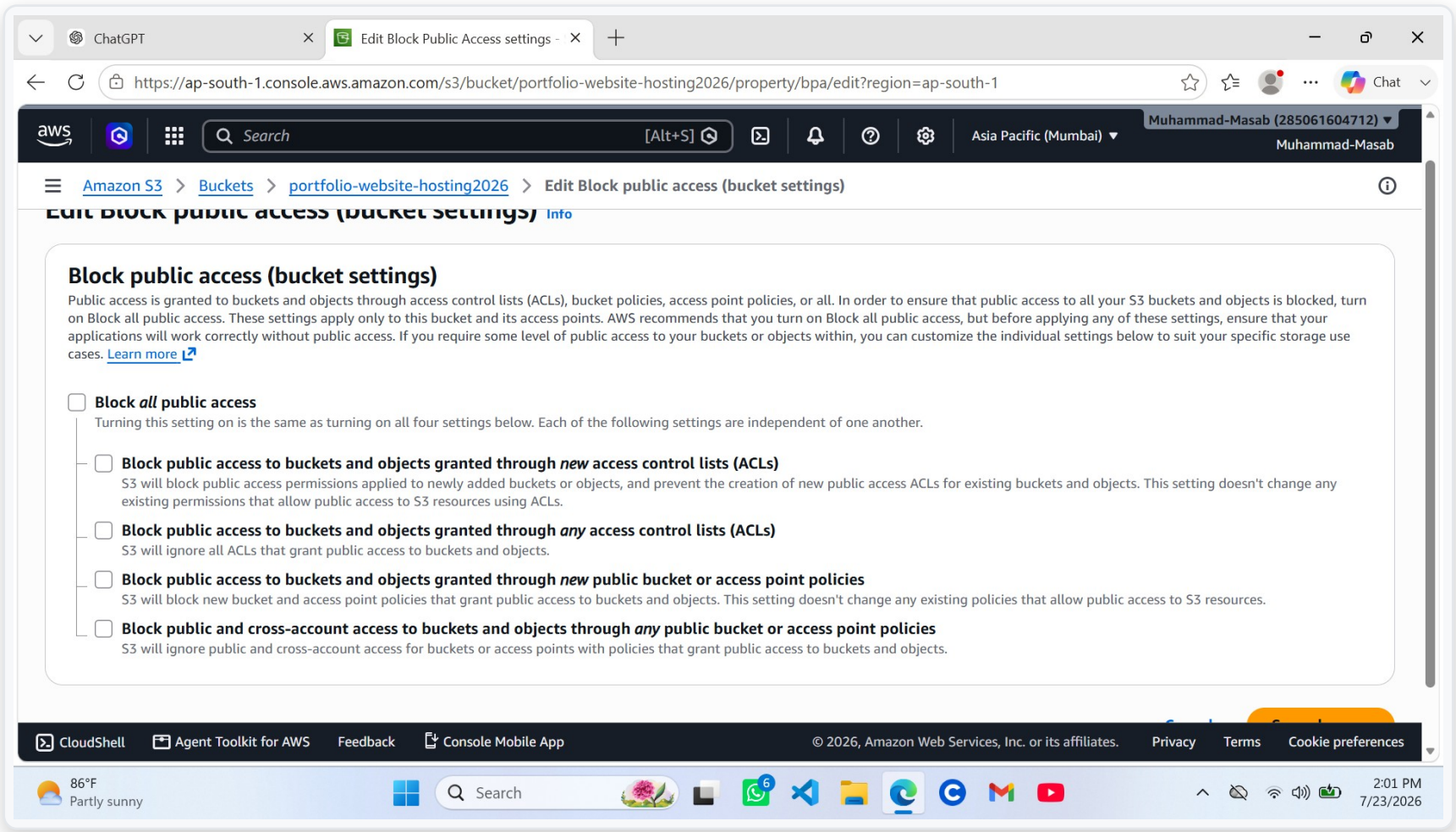
This is the switch that turns a plain storage bucket into an actual web server capable of responding to HTTP requests.

HOW

Bucket → Properties → Static website hosting → Edit → Enable → Host a static website → set the index document.

STEP 5 OF 5

Opening Up Public Access



WHAT

Edited Block Public Access (bucket settings) and unchecked all four restrictive toggles.

WHY

S3 blocks all public access by default. A public website needs visitors to read files without signing in to AWS.

HOW

Bucket → Permissions → Block Public Access → Edit → unchecked all four options → saved and confirmed.

This step alone doesn't make files public — it only allows a bucket policy to grant that access, which is exactly what came next.

THE FIX

A Bucket Policy — and a 404 I Didn't Expect

PUBLIC READ BUCKET POLICY

```
{ "Effect": "Allow", "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::portfolio-website-hosting2026/*" }
```

WHAT WENT WRONG

Loading the site returned a 404 NoSuchKey error for index.html. The cause: my files were sitting inside a "static website/" subfolder instead of the bucket root — so their real key was static website/index.html, not index.html.

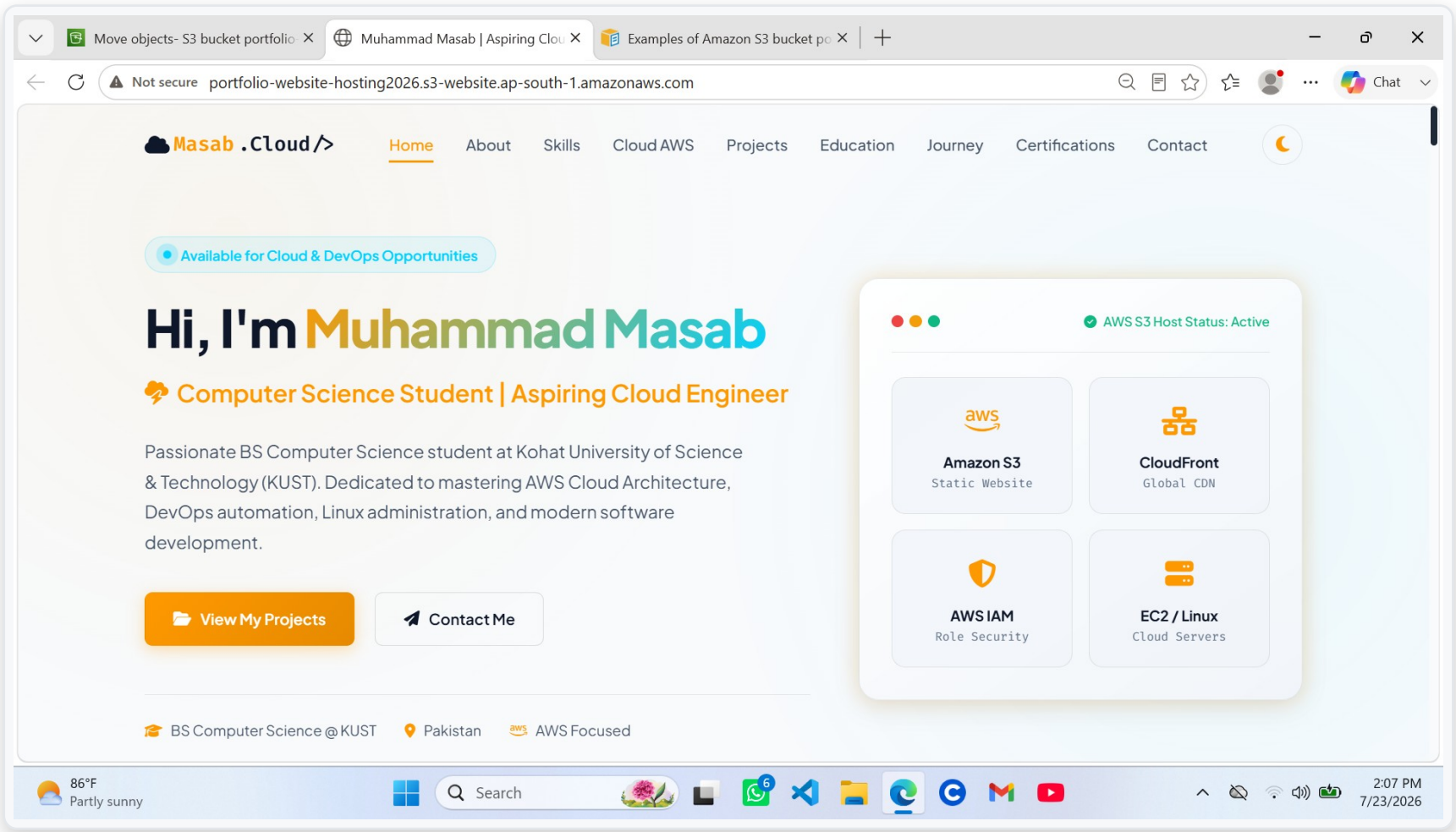
THE FIX

Moved index.html, style.css, and script.js to the bucket root so the object keys matched what static hosting expected — reloaded the endpoint, and the homepage rendered perfectly.

💡 *Lesson: S3 serves the index document from the bucket root — not from a nested folder — unless you explicitly configure it otherwise.*

✅ LIVE ON THE WEB

The Deployed Portfolio Website



WHAT THIS SHOWS

The site loads directly from the S3 website endpoint (portfolio-website-hosting2026.s3-website.ap-south-1.amazonaws.com) — no server, no hosting provider, just a public S3 bucket serving index.html, style.css, and script.js on request.

RESULT

Live, Public, and Deployed Entirely on S3.

No servers to patch. No infrastructure to manage. Just a bucket, a policy, and a static website endpoint — for a few cents a month.

- S3 static hosting = a full web server, no EC2 required
- Block Public Access + a bucket policy work together, not alone
- Object key paths matter — root vs. subfolder changes everything

Follow along as I keep building on AWS ☁️